

Soft Computing — Unit 3: Neural Networks

Syllabus: Introduction to Neural Networks (advent of modern neuroscience, classical AI vs NN, biological neurons & ANN, model of artificial neuron); Learning Methods (Hebbian, competitive, Boltzmann, etc.); Neural Network models (Perceptron, Adaline, Madaline, single-layer, back-propagation & multi-layer networks); Competitive learning networks (Kohonen SOM, Hebbian learning, Hopfield network); Neuro-Fuzzy modelling; Applications (pattern recognition & classification).

Table of Contents

1. [Introduction to Neural Networks](#)
2. [Biological Neuron vs Artificial Neuron](#)
3. [Model of an Artificial Neuron](#)
4. [Learning Methods in ANN](#)
5. [Single-Layer Networks — Perceptron, Adaline, Madaline](#)
6. [Multi-Layer Networks & Back-Propagation](#)
7. [Competitive Learning Networks — Kohonen SOM, Hebbian, Hopfield](#)
8. [Neuro-Fuzzy Modelling \(ANFIS\)](#)
9. [Applications — Pattern Recognition & Classification](#)
10. [Quick Revision Cheat-Sheet](#)

1. Introduction to Neural Networks



1.1 Advent of Modern Neuroscience

Year	Milestone
1873	Camillo Golgi — silver staining lets neurons be seen under microscope.

1888	Santiago Ramón y Cajal — <i>neuron doctrine</i> : brain is made of discrete cells.
1943	McCulloch & Pitts — first mathematical model of a neuron.
1949	Donald Hebb — " <i>Cells that fire together, wire together</i> " → Hebbian learning.
1958	Frank Rosenblatt — Perceptron.
1960	Widrow & Hoff — Adaline (LMS rule).
1969	Minsky & Papert — proved XOR limitation → "AI winter".
1982	Hopfield — recurrent associative memory.
1982	Kohonen — Self-Organizing Map (SOM).
1986	Rumelhart, Hinton, Williams — Back-Propagation revived MLPs.
1997	Hochreiter & Schmidhuber — LSTM.
2012	Krizhevsky et al. — AlexNet (CNN) wins ImageNet → deep-learning era.

Brain stats: $\approx 10^{11}$ neurons, $\approx 10^{14}$ – 10^{15} synapses, ~ 20 W power. Inspires *massively parallel, fault-tolerant* computing.

1.2 Classical AI vs Neural Networks

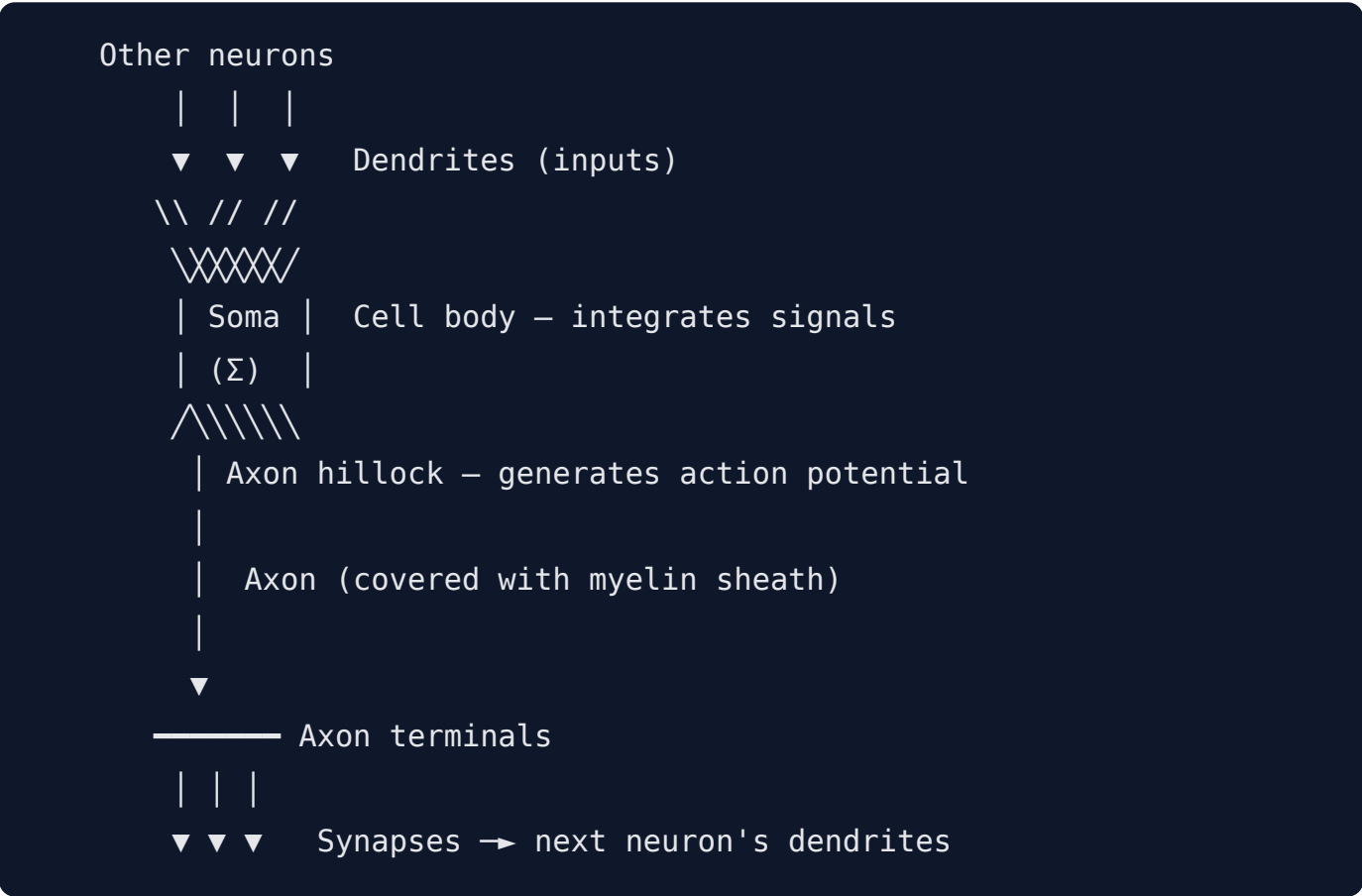
Aspect	Classical (Symbolic) AI	Neural Networks
Knowledge	Explicit symbols, rules	Distributed in weights
Reasoning	Deductive, logical	Inductive, pattern-based
Programming	Hand-crafted rules (expert systems)	Learns from data
Brittleness	Fails on noisy / novel inputs	Graceful degradation
Examples	Prolog, expert systems, theorem provers	MLP, CNN, RNN, Transformer
Interpretability	High (rules visible)	Low (black-box)

Hardware	Sequential CPU	Inherently parallel (GPU/TPU)
Strength	Discrete, structured problems	Perception, pattern recognition

Soft computing treats them as **complementary** → hybrid neuro-symbolic & neuro-fuzzy systems.

2. Biological Neuron vs Artificial Neuron

2.1 Biological Neuron Anatomy



Part	Role
Dendrites	Receive incoming electrochemical signals
Soma	Sums inputs; fires action potential if threshold crossed
Axon hillock	Decision point for "spike" generation
Axon	Long fiber carrying output spike

Myelin sheath	Insulator → faster signal (saltatory conduction)
Synapse	Chemical / electrical junction; strength = "weight"
Neurotransmitters	Chemicals released across synapse (excitatory / inhibitory)

- **All-or-none law:** Either fires fully, or not at all.
- **Plasticity:** Synaptic strengths change with experience → learning.

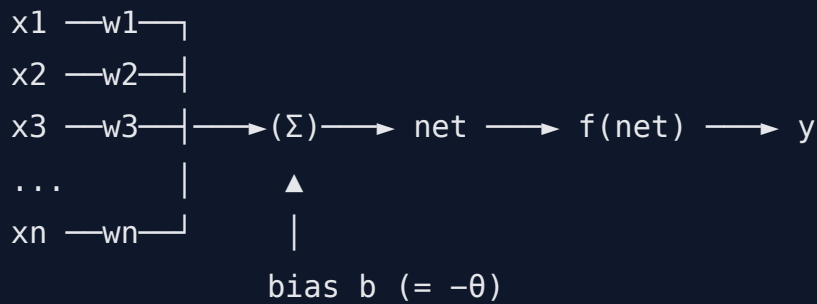
2.2 Mapping Biology → Artificial Neuron

Biological	Artificial
Dendrite	Input line x_i
Synapse	Weight w_i
Soma	Summing junction Σ
Action potential threshold	Activation function $f(\cdot)$
Axon	Output y
Synaptic plasticity	Weight update Δw

Property	Biological	Artificial
Number	$\approx 10^{11}$	$10^2 - 10^9$
Speed	ms (slow chemicals)	ns/ μ s (fast electronics)
Connections	$\approx 10^4$ per neuron	typically $10^2 - 10^3$
Power	≈ 20 W	kW–MW (GPU farms)
Robustness	High (neurons die daily, brain still works)	Improving (dropout, redundancy)

3. Model of an Artificial Neuron

3.1 Generic Neuron Model



Net input:

$$\text{net} = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^\top \mathbf{x} + b$$

Output:

$$y = f(\text{net})$$

Bias b shifts activation; equivalent to adding a constant "1" input with weight b .

3.2 McCulloch–Pitts Neuron (1943)

- **Binary inputs/outputs** $\in \{0, 1\}$.
- **Fixed weights** (no learning).
- **Threshold logic:** $y = 1$ if $\sum w_i x_i \geq \theta$, else 0.
- Can implement AND, OR, NOT — hence any Boolean function (by combining layers).

Example — OR Gate

$w_1 = w_2 = 1$, $\theta = 1 \rightarrow (0, 0) \rightarrow 0$, others $\rightarrow 1$. 

3.3 Activation Functions (Recap & Derivatives)

Name	$f(x)$	$f'(x)$	Range
Step	$1[x \geq 0]$	0 (undefined at 0)	$\{0, 1\}$
Signum	$\text{sgn}(x)$	0	$\{-1, 0, 1\}$

Linear	x	1	$\mathbb{R}$
Sigmoid	$\frac{1}{1 + e^{-x}}$	$f(x)(1 - f(x))$	$(0, 1)$
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	$1 - f(x)^2$	$(-1, 1)$
ReLU	$\max(0, x)$	$1[x > 0]$	$[0, \infty)$
Leaky ReLU	$\max(\alpha x, x), \alpha \approx 0.01$	α or 1	$\mathbb{R}$
Softmax	$\frac{e^{x_i}}{\sum_j e^{x_j}}$	$f_i(\delta_{ij} - f_j)$	$(0, 1)$

Sigmoid was the workhorse for back-prop. Modern deep nets prefer ReLU (avoids vanishing-gradient).

4. Learning Methods in ANN

4.1 Learning Paradigms

Paradigm	Feedback	Example algorithms
Supervised	Target output given	Perceptron, Adaline, BP
Unsupervised	No target — finds structure	Hebbian, SOM, Competitive
Reinforcement	Reward/penalty signal	Q-learning, TD-learning

General weight-update template:

$$w_{ij}(t + 1) = w_{ij}(t) + \Delta w_{ij}$$



4.2 Learning Rules — Catalogue

A. Hebbian Learning (1949)

"Cells that fire together, wire together."

$$\Delta w_{ij} = \eta x_i y_j$$

- Unsupervised. No bound $\rightarrow$ weights blow up unless normalized.
- **Oja's rule** (stable variant): $\Delta w_{ij} = \eta y_j (x_i - y_j w_{ij})$.

B. Perceptron Rule (Rosenblatt, 1958)

For binary classifier with target $t \in \{0, 1\}$ or $\{-1, +1\}$:

$$\Delta w_i = \eta(t - y)x_i, \quad \Delta b = \eta(t - y)$$

Only updates on misclassification.

C. Delta / Widrow-Hoff / LMS Rule (1960)

Minimizes squared error $E = \frac{1}{2}(t - y)^2$ for **linear** units:

$$\Delta w_i = \eta(t - y)x_i$$

For non-linear f : include derivative — $\Delta w_i = \eta(t - y)f'(\text{net})x_i$ (Generalized Delta).

D. Competitive (Winner-Take-All)

Only the neuron with largest activation updates:

$$\Delta w_{ij} = \begin{cases} \eta(x_i - w_{ij}) & j = j^* \\ 0 & \text{otherwise} \end{cases}$$

Used in SOM, LVQ, k-means clustering.

E. Correlation Rule

Supervised variant of Hebbian using target:

$$\Delta w_{ij} = \eta x_i t_j$$

F. Outstar (Grossberg) Rule

Moves weights toward target output:

$$\Delta w_{ij} = \eta(t_j - w_{ij})$$

G. Boltzmann Learning (1985, Hinton & Sejnowski)

Stochastic, energy-based learning for **Boltzmann machines** (symmetric, recurrent, binary stochastic neurons).

Energy of state $\mathbf{s}$:

$$E(\mathbf{s}) = -\frac{1}{2} \sum_{i \neq j} w_{ij} s_i s_j - \sum_i \theta_i s_i$$

Boltzmann–Gibbs probability of state:

$$P(\mathbf{s}) = \frac{e^{-E(\mathbf{s})/T}}{Z}, \quad Z = \sum_{\mathbf{s}'} e^{-E(\mathbf{s}')/T}$$

Learning rule — minimize KL divergence between *clamped* (data) and *free-running* phases:

$$\Delta w_{ij} = \frac{\eta}{T} (\langle s_i s_j \rangle_{\text{clamped}} - \langle s_i s_j \rangle_{\text{free}})$$

- T is *temperature* — annealed from high to low (simulated annealing).
- Slow but principled; basis of *Restricted Boltzmann Machines* (RBM) used in early deep learning.

Summary Comparison

Rule	Type	Update
Hebbian	Unsupervised	$\eta x_i y_j$
Perceptron	Supervised (discrete)	$\eta(t - y)x_i$
Delta / LMS	Supervised (continuous)	$\eta(t - y)f'(net)x_i$
Competitive	Unsupervised	$\eta(x_i - w_{ij})$ (winner only)
Outstar	Supervised	$\eta(t_j - w_{ij})$
Boltzmann	Stochastic	$\eta(\langle s_i s_j \rangle_+ - \langle s_i s_j \rangle_-)$

5. Single-Layer Networks

5.1 Perceptron (Rosenblatt, 1958)

Architecture

- One layer of TLU (threshold logic units).
- Activation: step.



Algorithm

```

initialize  $w, b \leftarrow$  small random / 0
repeat until no error:
  for each  $(x, t)$  in training set:
     $y = \text{step}(w \cdot x + b)$ 
     $w \leftarrow w + \eta(t - y)x$ 
     $b \leftarrow b + \eta(t - y)$ 

```

Perceptron Convergence Theorem (Novikoff, 1962)

If training data is **linearly separable**, the perceptron rule converges in finite steps. Bound:

$$\text{updates} \leq \left(\frac{R}{\gamma} \right)^2$$

where $R = \max \|\mathbf{x}\|$ and γ = margin of best separator.

XOR Limitation (Minsky-Papert 1969)

XOR is not linearly separable $\Rightarrow$ single perceptron **cannot** solve it; need a hidden layer.

Solved Problem — Train Perceptron for AND

Inputs $(x_1, x_2) \in \{0, 1\}^2$, target $t = x_1 \wedge x_2$. $\eta = 1$.

Initial: $w_1 = 0, w_2 = 0, b = 0$.

Step	(x_1, x_2)	t	y	$\Delta w_1, \Delta w_2, \Delta b$	w_1, w_2, b
1	(0,0)	0	1 (since $\text{net}=0 \geq 0$)	(0,0,-1)	(0,0,-1)
2	(0,1)	0	0	(0,0,0)	(0,0,-1)
3	(1,0)	0	0	(0,0,0)	(0,0,-1)

4	(1,1)	1	0	(1,1,1)	(1,1,0)
5	(0,0)	0	1 (net=0)	(0,0,-1)	(1,1,-1)
6	(0,1)	0	0	(0,0,0)	(1,1,-1)
7	(1,0)	0	0	(0,0,0)	(1,1,-1)
8	(1,1)	1	1 (net=1 $\geq$ 0)	(0,0,0)	(1,1,-1)

One more epoch with no errors $\Rightarrow$ **converged**: $w = (1, 1)$, $b = -1$. 

5.2 Adaline (Widrow & Hoff, 1960)

ADaptive LInear NEuron. Same architecture as a perceptron but:

- Uses **linear activation** (during training): $y = \text{net}$.
- Trained with **LMS (Least Mean Square)** rule.
- Output thresholded **after** training for classification.

Cost & Update

$$E(\mathbf{w}) = \frac{1}{2}(t - \text{net})^2, \quad \Delta w_i = \eta(t - \text{net})x_i$$

Difference from Perceptron

	Perceptron	Adaline
Activation during training	Step	Linear
Error	$t - y$ (binary)	$t - \text{net}$ (continuous)
Convergence	Only if linearly separable	LMS converges if η small enough; minimizes MSE
Output	Binary	Continuous (thresholded later)

Solved Problem — One Step of LMS

Initial $w = (0.2, -0.1)$, $b = 0.0$. Sample $(x_1, x_2) = (1, 1)$, $t = 1$, $\eta = 0.3$.

$\text{net} = 0.2 - 0.1 + 0 = 0.1$; error $e = 1 - 0.1 = 0.9$.

$\Delta w_1 = 0.3(0.9)(1) = 0.27$, $\Delta w_2 = 0.27$, $\Delta b = 0.27$.

New weights: $w = (0.47, 0.17)$, $b = 0.27$.

5.3 Madaline (Many Adalines)

Many Adaline units → one of the **earliest multi-layer** trainable networks.

- **MADALINE Rule I (MR-I, 1962)**: Outputs combined via fixed logic (e.g., majority). Adapts weights of the Adaline whose net is **closest to zero** when output is wrong.
- **MADALINE Rule II (MR-II, 1988)**: Generalised — flips outputs of units whose change most reduces error.
- **MADALINE Rule III (MR-III)**: Continuous-output version; precursor to back-prop.

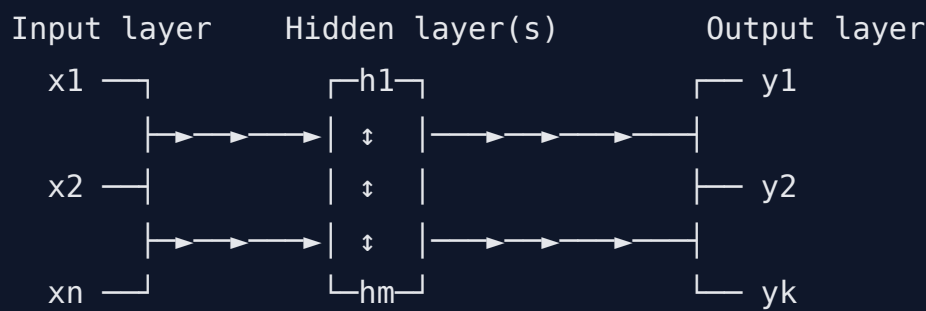
Architecture

```
x → Adaline1 ┘  
x → Adaline2 ┣→ Majority/AND/OR → y  
x → Adaline3 ┙
```

Madaline solves XOR (using ≥ 2 Adalines + AND/OR combiner) — overcoming perceptron limitation.

6. Multi-Layer Networks & Back-Propagation

6.1 MLP Architecture



- Fully connected; feed-forward.
- Each hidden/output neuron: $y_j = f(\sum_i w_{ij}x_i + b_j)$.

Universal Approximation Theorem (Cybenko 1989; Hornik 1991)

A feed-forward net with **one hidden layer** of a finite number of neurons, using a **non-constant, bounded, continuous** activation function (e.g., sigmoid), can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary precision.

6.2 Back-Propagation Algorithm (Rumelhart, Hinton, Williams 1986)

Notation

- Layer indices $\ell = 1, \dots, L$. Activations a^ℓ , pre-activations $z^\ell = W^\ell a^{\ell-1} + b^\ell$, $a^\ell = f(z^\ell)$.
- Loss for sample: $E = \frac{1}{2} \|t - a^L\|^2$ (or cross-entropy).
- Learning rate η .

Forward Pass

$$z^\ell = W^\ell a^{\ell-1} + b^\ell, \quad a^\ell = f(z^\ell)$$

Backward Pass

Output layer error:

$$\delta^L = (a^L - t) \odot f'(z^L)$$

Hidden layer error (backpropagation):

$$\delta^\ell = ((W^{\ell+1})^\top \delta^{\ell+1}) \odot f'(z^\ell)$$

Gradient w.r.t. weights & biases:

$$\frac{\partial E}{\partial W^\ell} = \delta^\ell (a^{\ell-1})^\top, \quad \frac{\partial E}{\partial b^\ell} = \delta^\ell$$

Weight update (gradient descent):

$$W^\ell \leftarrow W^\ell - \eta \delta^\ell (a^{\ell-1})^\top$$

Algorithm (Pseudo-code)

```

for epoch = 1..E:
    for (x, t) in training data:
        # Forward
        a0 ← x
        for ℓ = 1..L:
            zℓ = Wℓ · a(ℓ-1) + bℓ
            aℓ = f(zℓ)
        # Backward
        δL = (aL - t) ⊙ f'(zL)
        for ℓ = L-1..1:
            δℓ = (W(ℓ+1)⊤ · δ(ℓ+1)) ⊙ f'(zℓ)
        # Update
        for ℓ = 1..L:
            Wℓ ← Wℓ - η · δℓ · a(ℓ-1)⊤
            bℓ ← bℓ - η · δℓ

```

Variants

- **Batch / Mini-batch / Stochastic** gradient descent
- **Momentum:** $v \leftarrow \mu v - \eta \nabla E, W \leftarrow W + v$
- **Adam, RMSProp, AdaGrad** — adaptive learning rates
- **Regularization:** L2 weight decay, dropout, early stopping

Issues

Problem	Mitigation
Vanishing gradient (sigmoid/tanh)	ReLU, batch-norm, residual connections
Exploding gradient	Gradient clipping
Overfitting	Dropout, L2, more data, early stopping

Local minima / saddle points	Momentum, Adam, random restarts
Slow convergence	Adaptive optimizers

6.3 Solved Problem — One BP Step (2-2-1 Sigmoid Net)

Network: 2 inputs, 2 hidden, 1 output. Sigmoid activations.

Weights:

$$W^1 = \begin{bmatrix} 0.15 & 0.20 \\ 0.25 & 0.30 \end{bmatrix}, b^1 = \begin{bmatrix} 0.35 \\ 0.35 \end{bmatrix}, W^2 = \begin{bmatrix} 0.40 & 0.45 \end{bmatrix}, b^2 = 0.60$$

Input $x = (0.05, 0.10)$, target $t = 0.01$, $\eta = 0.5$.

Forward

$$z_1^1 = 0.15(0.05) + 0.20(0.10) + 0.35 = 0.3775 \Rightarrow a_1^1 = \sigma(0.3775) = 0.5933$$

$$z_2^1 = 0.25(0.05) + 0.30(0.10) + 0.35 = 0.3925 \Rightarrow a_2^1 = \sigma(0.3925) = 0.5969$$

$$z^2 = 0.40(0.5933) + 0.45(0.5969) + 0.60 = 1.1059 \Rightarrow a^2 = \sigma(1.1059) = 0.7514$$

$$\text{Error: } E = \frac{1}{2}(0.7514 - 0.01)^2 = 0.2749.$$

Backward

$$\delta^2 = (a^2 - t) \cdot a^2(1 - a^2) = (0.7414)(0.7514)(0.2486) = 0.1385$$

$$\delta_1^1 = \delta^2 \cdot W_{11}^2 \cdot a_1^1(1 - a_1^1) = 0.1385(0.40)(0.5933)(0.4067) = 0.01337$$

$$\delta_2^1 = 0.1385(0.45)(0.5969)(0.4031) = 0.01500$$

Update (one step)

$$\Delta W_{11}^2 = -\eta \delta^2 a_1^1 = -0.5(0.1385)(0.5933) = -0.0411$$

$$W_{11}^2 \leftarrow 0.40 - 0.0411 = \mathbf{0.3589}$$

$$W_{12}^2 \leftarrow 0.45 - 0.5(0.1385)(0.5969) = \mathbf{0.4087}$$

$$\Delta W_{11}^1 = -\eta \delta_1^1 x_1 = -0.5(0.01337)(0.05) = -0.000334$$

$$W_{11}^1 \leftarrow 0.15 - 0.000334 \approx \mathbf{0.14967}$$

Repeating drives $E \rightarrow 0$.

7. Competitive Learning Networks

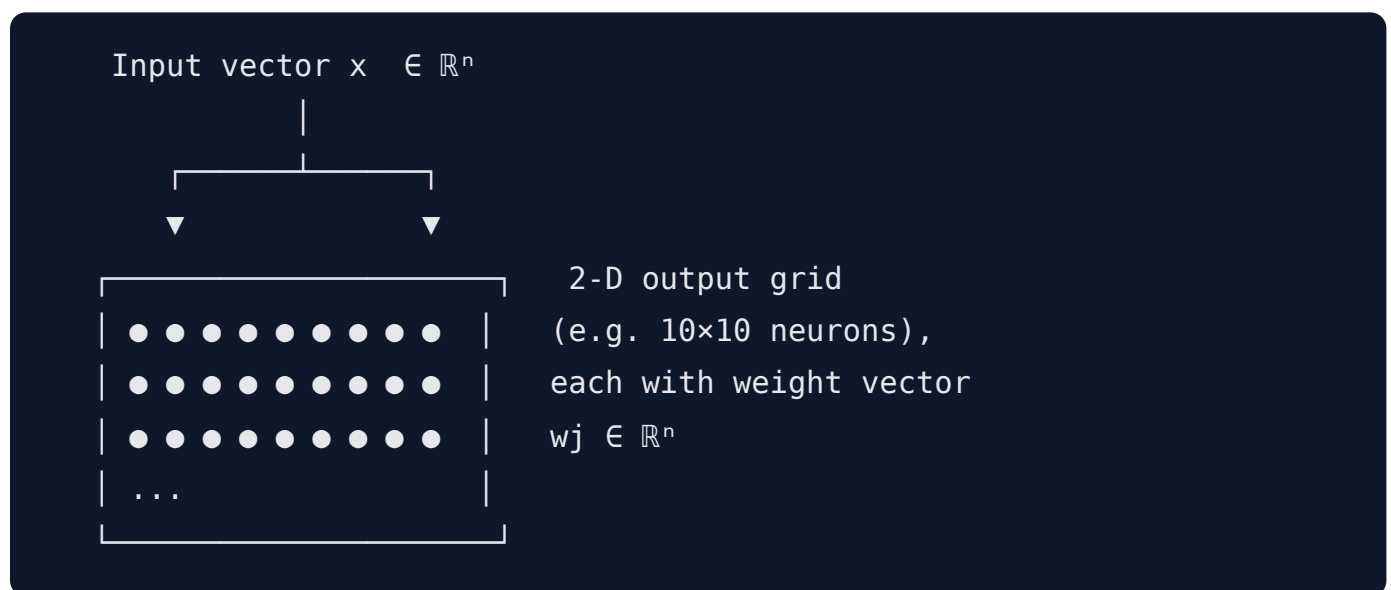
7.1 Competitive Learning (Winner-Take-All)

- All neurons compete for activation; **only one wins** per input.
- Winner: $j^* = \arg \max_j \mathbf{w}_j \cdot \mathbf{x}$ (or $\arg \min \|\mathbf{w}_j - \mathbf{x}\|$).
- **Update only winner:** $\mathbf{w}_{j^*} \leftarrow \mathbf{w}_{j^*} + \eta(\mathbf{x} - \mathbf{w}_{j^*})$.
- Used for **clustering, vector quantization, feature mapping**.

7.2 Kohonen Self-Organizing Map (SOM, 1982)

A **2-D grid** of neurons that maps high-dimensional input data to a low-dimensional **topologically ordered** map.

Architecture



Algorithm

```

initialize wj randomly, set initial  $\eta_0$  and neighborhood radius  $\sigma_0$ 
for t = 1..T:
    pick input x(t)
    # 1. Find Best Matching Unit (BMU)
     $j^* = \operatorname{argmin}_j \|x - w_j\|$ 
    # 2. Update BMU and neighbors (Gaussian neighborhood)
    for each neuron j in grid:
         $h_j = \exp(-\|r_j - r_{j^*}\|^2 / (2\sigma(t)^2))$ 
         $w_j \leftarrow w_j + \eta(t) \cdot h_j \cdot (x - w_j)$ 
    # 3. Decay learning rate  $\eta(t)$  and  $\sigma(t)$ 

```

Decay Schedules

$$\eta(t) = \eta_0 \exp(-t/\tau_\eta), \quad \sigma(t) = \sigma_0 \exp(-t/\tau_\sigma)$$

Properties

- **Topology preserving** — similar inputs activate nearby neurons.
- Unsupervised, used for dimensionality reduction, visualization, clustering (color quantization, document maps).

Mini Example

2-D inputs, 4-neuron SOM. Inputs cluster into 2 groups → neurons self-organize so 2 sit on each cluster, with adjacent neurons covering nearby points.

7.3 Hebbian Learning in NN

Classical Hebbian

$$\Delta w_{ij} = \eta x_i y_j$$

Weights grow without bound — causes instability.

Oja's Rule (1982) — Normalized Hebbian

$$\Delta w_{ij} = \eta y_j (x_i - y_j w_{ij})$$

Drives weight vector to the **principal eigenvector** of the input covariance matrix → **PCA**.

Sanger's Rule (Generalised Hebbian Algorithm)

Extracts the **first k principal components**:

$$\Delta w_{ij} = \eta y_j \left(x_i - \sum_{k \leq j} y_k w_{ik} \right)$$

7.4 Hopfield Network (1982)

A **single-layer, fully connected, recurrent** auto-associative memory.

Architecture

- N binary neurons, $s_i \in \{-1, +1\}$ (or $\{0, 1\}$).
- Symmetric weights $w_{ij} = w_{ji}$, $w_{ii} = 0$.
- Asynchronous update (one neuron at a time).



(fully connected, symmetric)

Update Rule

$$s_i \leftarrow \text{sgn} \left(\sum_{j \neq i} w_{ij} s_j - \theta_i \right)$$

Storage (Hebbian) — train on P patterns ξ^μ :

$$w_{ij} = \frac{1}{N} \sum_{\mu=1}^P \xi_i^\mu \xi_j^\mu, \quad i \neq j$$



Energy Function

$$E = -\frac{1}{2} \sum_{i \neq j} w_{ij} s_i s_j + \sum_i \theta_i s_i$$

Theorem (Hopfield): With symmetric weights and asynchronous updates, E is **non-increasing** at each step $\Rightarrow$ the network converges to a local minimum (stable state) in **finite** time.

Capacity

- Hebbian storage capacity $\approx 0.138N$ patterns (for low error).
- More patterns $\Rightarrow$ **spurious states** appear.

Uses

- Content-addressable memory (recall full pattern from noisy/partial).
- Optimization (e.g., **Travelling Salesman**, by encoding constraints in E).



Solved Problem — Store 1 Pattern

$N = 4$, pattern $\xi = (+1, -1, +1, -1)$. Then

$$w_{ij} = \frac{1}{4}\xi_i\xi_j \quad (i \neq j)$$

$$W = \frac{1}{4} \begin{bmatrix} 0 & -1 & 1 & -1 \\ -1 & 0 & -1 & 1 \\ 1 & -1 & 0 & -1 \\ -1 & 1 & -1 & 0 \end{bmatrix}$$

Test with noisy input $s = (+1, +1, +1, -1)$ (bit 2 flipped). Update neuron 2:

$$\text{net}_2 = -\frac{1}{4}(1) + (-\frac{1}{4})(1) + \frac{1}{4}(-1) = -0.75 < 0 \Rightarrow s_2 = -1 \quad \checkmark$$

Pattern recovered.

8. Neuro-Fuzzy Modelling (ANFIS)



8.1 Motivation

Fuzzy Logic	Neural Networks
Good at incorporating expert knowledge	Good at learning from data
Hard to learn rules / MFs automatically	Black-box, no interpretability

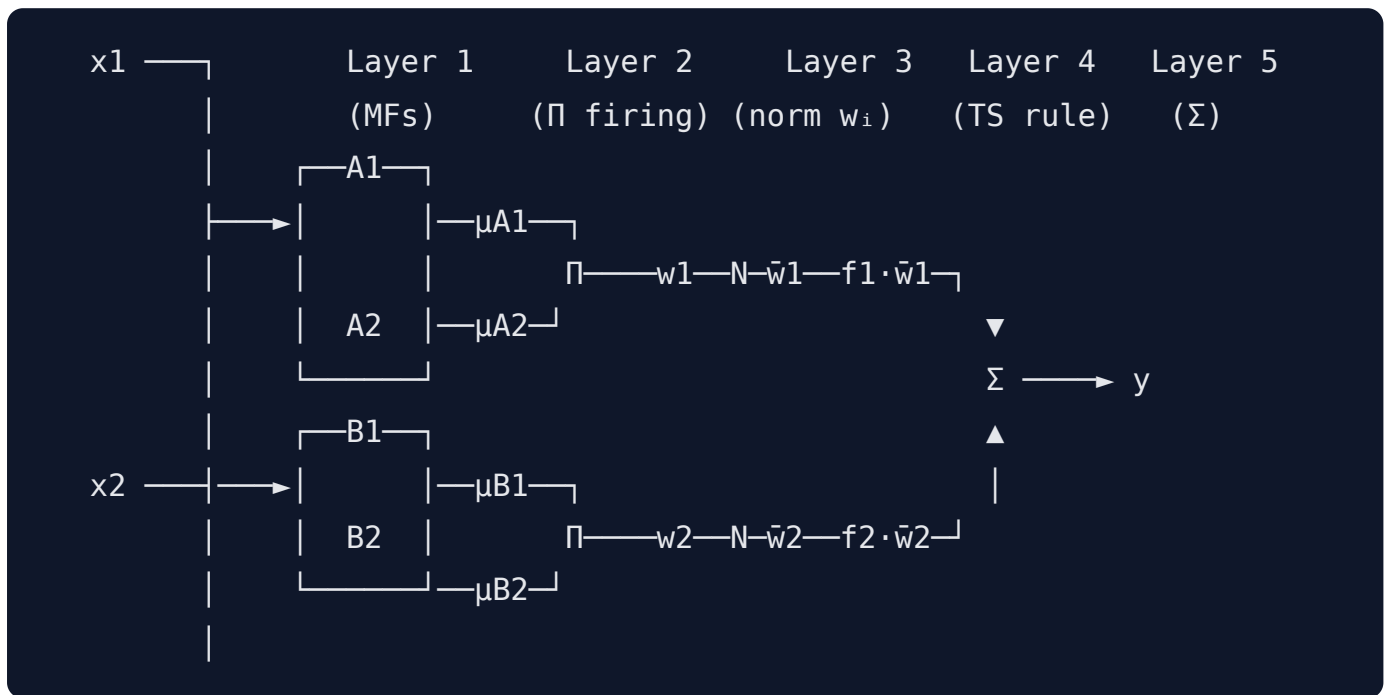
Neuro-Fuzzy combines both:

- Fuzzy framework (interpretable, linguistic).

- Neural learning (data-driven MF & rule tuning).

8.2 ANFIS — Adaptive Neuro-Fuzzy Inference System (Jang, 1993)

Implements a **first-order Sugeno FIS** as a 5-layer feed-forward network.



Layer-wise functions

Layer 1 — Adaptive (MF parameters):

$$O_i^1 = \mu_{A_i}(x)$$

Typically bell-shaped:

$$\mu_{A_i}(x) = \frac{1}{1 + \left| \frac{x - c_i}{a_i} \right|^{2b_i}}$$

Layer 2 — Fixed (rule firing w_i):

$$O_i^2 = w_i = \mu_{A_i}(x_1) \cdot \mu_{B_i}(x_2)$$

Layer 3 — Fixed (normalization):

$$O_i^3 = \bar{w}_i = \frac{w_i}{\sum_k w_k}$$

Layer 4 — Adaptive (consequent params):

$$O_i^4 = \bar{w}_i f_i = \bar{w}_i (p_i x_1 + q_i x_2 + r_i)$$

Layer 5 — Fixed (sum):

$$y = \sum_i \bar{w}_i f_i = \frac{\sum_i w_i f_i}{\sum_i w_i}$$

Hybrid Learning

- **Forward pass:** Fix antecedent params; estimate consequent params (p_i, q_i, r_i) by **Least-Squares**.
- **Backward pass:** Fix consequents; update antecedent params (a_i, b_i, c_i) by **back-propagation**.

→ Faster than pure BP, retains interpretability.

Other Neuro-Fuzzy Models

- **FALCON** (Fuzzy Adaptive Learning Control Network) — Lin & Lee, 1991.
- **NEFCON / NEFCLASS / NEFPROX** — Nauck & Kruse.
- **GARIC** — Berenji, 1992 (reinforcement learning + fuzzy).

9. Applications — Pattern Recognition & Classification

9.1 Pattern Recognition Pipeline

```

Raw data → Pre-processing → Feature extraction → Classifier →
Decision
              (noise removal,      (PCA, FFT, edges,      (NN, SVM,
(class label)          normalize)      wavelets)          RBF,
              Hopfield)

```

Stage	Typical techniques
Pre-processing	Normalization, denoising, segmentation

Feature extraction	PCA, LDA, Fourier, edge filters, CNN features
Classifier	MLP, RBF, SOM, Hopfield, deep CNN
Post-processing	Voting, smoothing, confidence thresholding

9.2 Classification with MLP

Setup

- K output neurons, one per class; **one-hot** target.
- Output uses **softmax** + **cross-entropy** loss:

$$P(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad E = - \sum_k t_k \log P(y = k \mid \mathbf{x})$$

Real-world examples

Domain	Task	Network
OCR / Handwriting	digit recognition (MNIST)	MLP, CNN (LeNet, AlexNet)
Speech	phoneme classification	TDNN, RNN, LSTM
Vision	face / object detection	CNN, ResNet, YOLO
Medical	tumor classification	CNN + transfer learning
Finance	credit scoring, fraud detection	MLP, autoencoder
Robotics	scene understanding	CNN + RL

9.3 Pattern Recognition with Specialized NN

Network	Recognition Role
Hopfield	Auto-associative — recall stored pattern from noisy input
SOM	Cluster & visualize high-D patterns; phoneme map

RBF	Function approximation, local-pattern classification
CNN	Spatial patterns (images)
RNN/LSTM	Temporal patterns (sequences, speech, language)
Autoencoder	Anomaly detection, denoising, dimensionality reduction

9.4 Solved Problem — Character Recognition with Hopfield

Goal: Store the 5×5 binary pixel patterns of letters "L" and "T", and recover them from noisy inputs.

1. Flatten each pattern $\rightarrow$ vector $\xi^\mu \in \{-1, +1\}^{25}$.
2. Compute weights:

$$w_{ij} = \frac{1}{25} \sum_{\mu=1}^2 \xi_i^\mu \xi_j^\mu, \quad i \neq j$$

3. Feed noisy "L" (some pixels flipped) $\rightarrow$ run asynchronous updates $\rightarrow$ converges to clean "L". 

Capacity $\approx 0.138 \times 25 \approx 3$ patterns; suitable here.

10. Quick Revision Cheat-Sheet



History

- 1943 McCulloch-Pitts $\rightarrow$ 1949 Hebb $\rightarrow$ 1958 Perceptron $\rightarrow$ 1969 XOR proof $\rightarrow$ 1982 Hopfield/SOM $\rightarrow$ 1986 Back-prop $\rightarrow$ 2012 Deep Learning.



Biological vs Artificial

- Dendrite=input, soma= Σ , axon=output, synapse=weight.
- Brain: $\sim 10^{11}$ neurons, $\sim 10^{14}$ synapses, 20 W.



Neuron Model

- $y = f(\sum w_i x_i + b)$.
- McCulloch-Pitts: binary, fixed weights.
- Activations: step, sigmoid ($\sigma' = \sigma(1 - \sigma)$), tanh, ReLU.



Learning Rules

- **Hebbian:** $\Delta w = \eta xy$.
- **Perceptron:** $\Delta w = \eta(t - y)x$ (binary).
- **Delta/LMS:** continuous targets, minimizes MSE.
- **Competitive:** $\Delta w = \eta(x - w)$ for winner.
- **Boltzmann:** stochastic, energy-based, $\Delta w \propto \langle s_i s_j \rangle_+ - \langle s_i s_j \rangle_-$.



Single-Layer Networks

- Perceptron — separates linearly, can't solve XOR.
- Adaline — linear activation, LMS rule.
- Madaline — multiple Adalines; solves XOR.



Back-Propagation

- Forward: $z^\ell = W^\ell a^{\ell-1} + b^\ell$, $a^\ell = f(z^\ell)$.
- $\delta^L = (a^L - t) \odot f'(z^L)$.
- $\delta^\ell = ((W^{\ell+1})^\top \delta^{\ell+1}) \odot f'(z^\ell)$.
- $\nabla_{W^\ell} E = \delta^\ell (a^{\ell-1})^\top$.
- Universal approximation: 1 hidden layer suffices (Cybenko 1989).



Competitive / Recurrent

- **SOM:** topology-preserving 2-D map. Find BMU, update neighborhood (decaying η , σ).
- **Hebbian/Oja:** PCA-like.
- **Hopfield:** symmetric recurrent, energy non-increasing, capacity $\approx 0.138 N$.



Neuro-Fuzzy / ANFIS

- 5-layer Sugeno-style FIS.
- Hybrid learning: LSE for consequents + BP for antecedents.



Applications

- Pattern recognition pipeline: preprocess → features → classify.
 - MLP + softmax + cross-entropy for K-class problems.
 - Hopfield = auto-associative recall, SOM = clustering, CNN = images, RNN/LSTM = sequences.
-

End of Unit 3 — Neural Networks.

Next: Unit 4 (share when ready — likely Genetic Algorithms / Hybrid Systems).